

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

```
main.c
1  /
2
3  8  #include <stdio.h>
4  9  #include <string.h>
5
6  10
7  11  int isPalindrome(char str[])
8  12  {
9  13      int left = 0;
10 14      int right = strlen(str) - 1;
11
12 15
13 16      while(left < right)
14 17      {
15 18          if(str[left] != str[right])
16 19              return 0;
17
18 20
19 21          left++;
20 22          right--;
21
22 23      }
23
24 24
25 25      return 1;
26
27 26
28 27
29 28  int main()
30 29  {
31 30      int n;
32
33 31
34 32      printf("Enter number of words: ");
35 33      scanf("%d", &n);
36
37 34
38 35      char words[n][100];
39
40 36
41 37      printf("Enter words:\n");
42
43 38
44 39      for(int i = 0; i < n; i++)
45 40          scanf("%s", words[i]);
```

```
Enter number of words: 5
Enter words:
abc ada acb
ar
akkf
First Palindromic String = ada
...Program finished with exit code 0
Press ENTER to exit console.
```

2. You are given two integer arrays `nums1` and `nums2` of sizes `n` and `m`, respectively. Calculate the following values: `answer1` : the number of indices `i` such that `nums1[i]` exists in `nums2`. `answer2` : the number of indices `i` such that `nums2[i]` exists in `nums1`. Return `[answer1,answer2]`.

Example 1:

Input: `nums1 = [2,3,2]`, `nums2 = [1,2]`

Output: `[2,1]`

Explanation:

Example 2:

Input: `nums1 = [4,3,2,3,1]`, `nums2 = [2,2,5,2,3,6]`

Output: `[3,4]`

Explanation:

The elements at indices 1, 2, and 3 in `nums1` exist in `nums2` as well. So `answer1` is 3.

The elements at indices 0, 1, 3, and 4 in `nums2` exist in `nums1`. So `answer2` is 4.

```
7  *****/
8  #include <stdio.h>
9
10 int exists(int arr[], int n, int value)
11 {
12     for(int i = 0; i < n; i++)
13     {
14         if(arr[i] == value)
15             return 1;
16     }
17     return 0;
18 }
19
20 int main()
21 {
22     int n, m;
23
24     printf("Enter size of nums1: ");
25     scanf("%d", &n);
26
27     int nums1[n];
28
29     printf("Enter elements of nums1: ");
30     for(int i = 0; i < n; i++)
31         scanf("%d", &nums1[i]);
32
33     printf("Enter size of nums2: ");
34     scanf("%d", &m);
35
36     int nums2[m];
37
38     printf("Enter elements of nums2: ");
39     for(int i = 0; i < m; i++)
```

```
input
Enter size of nums1: 3
Enter elements of nums1: 1 5 7
Enter size of nums2: 2
Enter elements of nums2: 7 9
Output: [1,1]

...Program finished with exit code 0
Press ENTER to exit console.
```

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to $12 + 12 + 12 + 22 + 22 + 22 = 15$.

Example 2:

Input: nums = [1,1]

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to $12 + 12 + 12 = 3$.

```
1 // *****
2
3 #include <stdio.h>
4
5 int main()
6 {
7     int n;
8
9     printf("Enter size of array: ");
10    scanf("%d", &n);
11
12    int nums[n];
13
14    printf("Enter array elements: ");
15    for(int i = 0; i < n; i++)
16        scanf("%d", &nums[i]);
17
18    int sum = 0;
19
20    for(int i = 0; i < n; i++)
21    {
22        int freq[1001] = {0};
23        int distinct = 0;
24
25        for(int j = i; j < n; j++)
26        {
27            if(freq[nums[j]] == 0)
28            {
29                distinct++;
30                freq[nums[j]] = 1;
31            }
32
33            sum += distinct * distinct;
34        }
35    }
36}
```

```
input
Enter size of array: 2
Enter array elements: 4 9
Sum of Squares of Distinct Counts = 6
...Program finished with exit code 0
Press ENTER to exit console.
```

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where $0 \leq i < j < n$, such that $\text{nums}[i] == \text{nums}[j]$ and $(i * j)$ is divisible by k.
- Example 1:

Input: nums = [3,1,2,2,2,1,3], k = 2

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- nums[0] == nums[6], and $0 * 6 == 0$, which is divisible by 2.
- nums[2] == nums[3], and $2 * 3 == 6$, which is divisible by 2.
- nums[2] == nums[4], and $2 * 4 == 8$, which is divisible by 2.
- nums[3] == nums[4], and $3 * 4 == 12$, which is divisible by 2.

Example 2:

Input: nums = [1,2,3,4], k = 1

Output: 0

Explanation: Since no value in nums is repeated, there are no pairs (i,j) that meet all the requirements.

```
8  #include <stdio.h>
9
10 int main()
11 {
12     int n, k;
13
14     printf("Enter size of array: ");
15     scanf("%d", &n);
16
17     int nums[n];
18
19     printf("Enter array elements: ");
20     for(int i = 0; i < n; i++)
21         scanf("%d", &nums[i]);
22
23     printf("Enter value of k: ");
24     scanf("%d", &k);
25
26     int count = 0;
27
28     for(int i = 0; i < n; i++)
29     {
30         for(int j = i + 1; j < n; j++)
31         {
32             if(nums[i] == nums[j] &&
33                ((i * j) % k == 0))
34             {
35                 count++;
36             }
37         }
38     }
39
40     printf("Number of Valid Pairs = %d\n", count);
```

```
input
Enter size of array: 2
Enter array elements: 5 6
Enter value of k: 1
Number of Valid Pairs = 0

...Program finished with exit code 0
Press ENTER to exit console.
```

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

- 1) Input: {1, 2, 3, 4, 5} Expected Output: 5
- 2) Input: {7, 7, 7, 7, 7} Expected Output: 7
- 3) Input: {-10, 2, 3, -4, 5} Expected Output: 5

```
*****  
#include <stdio.h>  
  
int findMax(int arr[], int n)  
{  
    int max = arr[0];  
  
    for(int i = 1; i < n; i++)  
    {  
        if(arr[i] > max)  
            max = arr[i];  
    }  
  
    return max;  
}  
  
int main()  
{  
    int arr[] = {1, 2, 3, 4, 5};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    printf("%d\n", findMax(arr, n));  
  
    return 0;  
}
```

```
input  
5  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List
 1. Input: []
 2. Expected Output: None or an appropriate message indicating that the list is empty.
2. Single Element List

1. Input: [5]
2. Expected Output: 5

```
main.c
14     for(int j = 0; j < n - i - 1; j++)
15     {
16         if(arr[j] > arr[j + 1])
17         {
18             int temp = arr[j];
19             arr[j] = arr[j + 1];
20             arr[j + 1] = temp;
21         }
22     }
23 }
24 }
25
26 int findMax(int arr[], int n)
27 {
28     if(n == 0)
29         return -1;
30
31     sort(arr, n);
32     return arr[n - 1];
33 }
34
35 int main()
36 {
37     int arr[] = {3, 3, 3, 3, 3};
38     int n = sizeof(arr) / sizeof(arr[0]);
39
40     if(n == 0)
41         printf("List is empty\n");
42     else
43         printf("Maximum element: %d\n", findMax(arr, n));
44
45     return 0;
46 }
```

input

```
Maximum element: 3

...Program finished with exit code 0
Press ENTER to exit console.
```


3. All Elements are the Same

1. Input: [3, 3, 3, 3, 3]
2. Expected Output: 3

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

- Input: [3, 7, 3, 5, 2, 5, 9, 2]
- Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

- Input: [-1, 2, -1, 3, 2, -2]
- Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

- Input: [1000000, 999999, 1000000]
- Expected Output: [1000000, 999999]

```
.c
Online C Compiler.
Code, Compile, Run and Debug C program online.
Write your code in this editor and press "Run" button to compile and execute it.

/******************************************************************/
#include <stdio.h>

int main()
{
    int arr[] = {3, 7, 3, 5, 2, 5, 9, 2};
    int n = sizeof(arr) / sizeof(arr[0]);

    int unique[n];
    int k = 0;

    for(int i = 0; i < n; i++)
    {
        int found = 0;

        for(int j = 0; j < k; j++)
        {
            if(arr[i] == unique[j])
            {
                found = 1;
                break;
            }
        }

        if(!found)
        {
            unique[k] = arr[i];
            k++;
        }
    }
}
```

input


```
Unique elements: 3 7 5 2 9

...Program finished with exit code 0
Press ENTER to exit console.
```

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

```
#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    int i, j, temp;

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main()
{
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    bubbleSort(arr, n);

    printf("Sorted array: ");
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

```
Sorted array: 11 12 22 25 34 64 90

..Program finished with exit code 0
Press ENTER to exit console.
```

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example $X = \{3, 4, 6, -9, 10, 8, 9, 30\}$ KEY=10

Output: Element 10 is found at position 5

Example $X = \{3, 4, 6, -9, 10, 8, 9, 30\}$ KEY=100

Output : Element 100 is not found

```
8
9  #include <stdio.h>
10
11 int binarySearch(int arr[], int n, int key)
12 {
13     int low = 0, high = n - 1;
14
15     while(low <= high)
16     {
17         int mid = (low + high) / 2;
18
19         if(arr[mid] == key)
20             return mid;
21
22         if(arr[mid] < key)
23             low = mid + 1;
24         else
25             high = mid - 1;
26     }
27
28     return -1;
29 }
30
31 int main()
32 {
33     int arr[] = {-9, 3, 4, 6, 8, 9, 10, 30};
34     int n = sizeof(arr) / sizeof(arr[0]);
35
36     int key = 10;
37
38     int result = binarySearch(arr, n, key);
39
40     if(result != -1)
```

```
input
Element 10 is found at position 7
..Program finished with exit code 0
Press ENTER to exit console.
www.onlinegdb.com/online_c_compiler
```

10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

```
#include <stdio.h>

void merge(int arr[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for(int i = 0; i < n1; i++)
        L[i] = arr[l + i];

    for(int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;

    while(i < n1 && j < n2)
    {
        if(L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while(i < n1)
        arr[k++] = L[i++];

    while(j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r)
```

```
Sorted Array: 1 2 3 5

..Program finished with exit code 0
Press ENTER to exit console.
```